

**Dica**

Rascunho isolado da Aula 4. A apostila completa (`gerar_apostila.py`) *não* muda. Quando este texto estiver pronto, ele entra no gerador principal.

# Sumário

Clique no título para ir à seção.

**AULA 4**

|                                                 |   |
|-------------------------------------------------|---|
| <b>Tabelas Hash (Dicionários/Mapas)</b>         | 3 |
| O problema (antes da técnica)                   | 3 |
| Resposta em uma frase                           | 3 |
| O dict do Python já é isso                      | 3 |
| Peças da tabela hash                            | 4 |
| A classe TabelaHash — por que criar             | 4 |
| Inserir e buscar (fluxo mental)                 | 5 |
| Colisão — o ponto que mais confunde             | 7 |
| Exercício na mão (antes do código)              | 8 |
| Live coding: tabela com chaining                | 8 |
| Quando o prédio fica pequeno — crescer a tabela | 9 |



## AULA 4

# Tabelas Hash (Dicionários/Mapas)

**Objetivo desta aula (2h30)**

Entender o porquê do hash, mapear chave → índice, tratar colisão por encadeamento e buscar em tempo médio  $O(1)$  — sem usar o `dict` nativo no catálogo da aula.

**Para seguir, use o VS Code**

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

**Instalação** (pág. **Erro! Indicador não definido.**)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. **Erro! Indicador não definido.**)

Gancho da Aula 3: o gerente quer o cliente “Carlos” sem percorrer a fila inteira. Array é  $O(1)$  por índice, mas o índice não é o CPF. Hash transforma a chave em índice.

## O problema (antes da técnica)

Com array, achar `dados[3]` é instantâneo ( $O(1)$ ): você já sabe o endereço. Na vida real a pergunta costuma ser outra: “qual o nome do usuário `u001`?” ou “qual o preço do produto `SKU-9981`?”. Aqui a chave *não* é o índice. Se você só tiver uma lista, precisa percorrer até achar →  $O(n)$ .

*Pergunta da aula:* como transformar uma chave qualquer (texto, id, CPF) em um índice rápido?

## Resposta em uma frase

*Hash* = função que transforma a chave em um número; esse número vira o índice de um “balde” na tabela. Depois disso, você só procura *dentro daquele balde*, não na tabela inteira.

Analogia do prédio: imagine 5 andares (baldes 0 a 4). Chega a moradora com a chave “`u001`”. O porteiro (função de hash) calcula: “ela mora no andar 2”. Você sobe *só* o andar 2 e procura o nome na listinha daquele andar. Se duas pessoas caírem no mesmo andar, as duas ficam na listinha — isso é *colisão*. Ainda assim você não vasculha o prédio inteiro.

## O dict do Python já é isso

```
peessoa = {"nome": "Bruna", "idade": "20"}
print(peessoa["nome"]) # rápido — o dict já é hash
```

O `dict` nativo é uma tabela hash (bem otimizada). Nesta aula você *não* usa o `dict` para guardar o catálogo: recria a ideia na mão, para entender o que o Python faz por baixo.

- ▶ `dict` do Python — hash pronto, uso diário.

- ▶ `TabelaHash` da aula — hash didático, para estudar o mecanismo.

## Peças da tabela hash

- ▶ *Tabela* — array de tamanho fixo (ex.: 5 ou 8 posições).
- ▶ *Balde* — cada posição é uma listinha (vazia no início).
- ▶ *Função de hash* — transforma a chave em inteiro.
- ▶ *Índice* — `hash(chave) % tamanho` (fica entre 0 e `tamanho - 1`).
- ▶ *Par* — dentro do balde guardamos `[chave, valor]`.

### Dica

*Balde* é o nome padrão (em inglês: *bucket*). Não chamamos só de “posição” porque cada índice é um *recipiente*: pode estar vazio, ter um par ou vários (quando há colisão). Na analogia, “andar” ajuda a imaginar; na estrutura, o nome técnico é balde. Outros textos usam *slot* (endereçamento aberto) ou *bin*.

```
tamanho = 5

baldes:
  [0] []
  [1] []
  [2] [ ["u001", "João"], ["outra", "..."] ] ← colisão
  [3] [ ["u002", "Maria"] ]
  [4] []
```

## A classe TabelaHash — por que criar

As peças (tamanho, baldes, porteiro) *andam juntas*. Se fossem variáveis soltas, cada função precisaria receber `baldes` e `tamanho` na mão. A *classe* é o molde do prédio: um só lugar guarda os andares e o porteiro que calcula o andar.

Por isso criamos `class TabelaHash` antes de `inserir` e `buscar`. Sem a classe não existe `self` — e sem `self` o porteiro não sabe quantos andares este prédio tem.

- ▶ `class TabelaHash` — o projeto do prédio (molde).
- ▶ `__init__` — roda quando o prédio é construído: define quantos andares e abre as listinhas vazias.
- ▶ `_hash` — o porteiro *deste* prédio. O `_` na frente = método interno (a classe usa; o aluno de fora chama `inserir` / `buscar`).
- ▶ `self` — “este objeto”. Quando você faz `catalogo.inserir(...)`, o Python coloca o `catalogo` no `self`.
- ▶ `catalogo = TabelaHash(5)` — um prédio de verdade, com 5 andares. Dá para ter outro: `produtos = TabelaHash(8)` — outro prédio, outros baldes.

esqueleto — ainda sem inserir/buscar

```
class TabelaHash:
    def __init__(self, tamanho=8):
        self.tamanho = tamanho
        self.baldes = [[] for _ in range(tamanho)]

    def _hash(self, chave):
        return sum(ord(c) for c in str(chave)) % self.tamanho

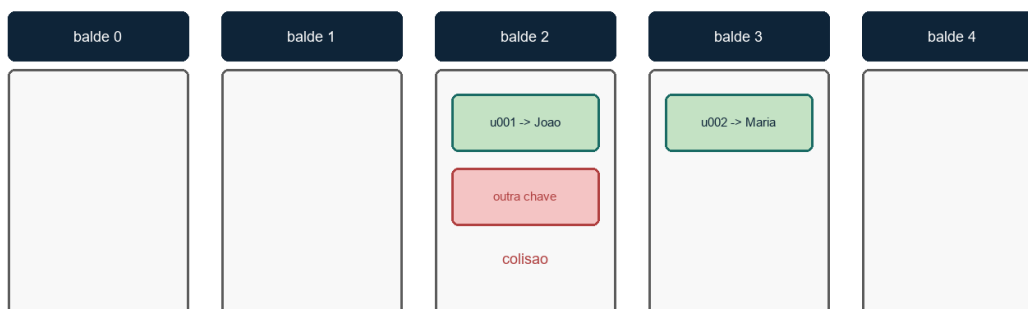
catalogo = TabelaHash(5)
```

Fórmula didática do curso, agora *dentro* da classe: `indice = (soma dos códigos dos caracteres) % self.tamanho`.

### Dica

`self._hash(chave)` (que vem no próximo bloco) é o `catalogo` chamando o *próprio* porteiro. `ord()` devolve o código do caractere; `sum()` soma; `%` cabe o índice na tabela. Essa função é só *para aprender* — a do `dict` real é mais sofisticada.

Tabela hash: `indice = hash(chave) % tamanho` + colisão por encadeamento



Cada balde é uma listinha. Duas chaves no mesmo índice = colisão; as duas ficam no encadeamento.

## Inserir e buscar (fluxo mental)

A classe já existe: `__init__` abriu os baldes e `_hash` é o porteiro. `inserir` e `buscar` usam `self._hash` porque estão *dentro do mesmo prédio*. Agora montamos esses dois métodos um passo de cada vez — o código cresce até ficar igual ao `exemplos/04_hash.py`.

Inserir "u001" → "João":

**Passo 1.** Calcular `indice = self._hash("u001")`.

O porteiro transforma a chave em andar. Com a função da aula, "u001" soma 262 e  $262 \% 5 = 2$  — balde 2.

inserir — passo 1

```
def inserir(self, chave, valor):
```

```
indice = self._hash(chave)
```

## Passo 2. Ir ao `baldes[indice]`.

Você sobe só aquele andar. O `for` percorre a listinha do balde 2 — os outros nem são abertos.

inserir — passo 2

```
def inserir(self, chave, valor):
    indice = self._hash(chave)
    for par in self.baldes[indice]:
        ...
```

## Passo 3. Se a chave *já existe* nesse balde → atualiza o valor (não duplica).

`par[0]` é a chave; `par[1]` é o valor. Se achar `"u001"` de novo, só troca o nome e dá `return` — não cria outro `par`.

inserir — passo 3

```
def inserir(self, chave, valor):
    indice = self._hash(chave)
    for par in self.baldes[indice]:
        if par[0] == chave:
            par[1] = valor
            return
```

## Passo 4. Se não existe → `append()` [`chave`, `valor`] na listinha.

O `for` acabou sem achar a chave: o andar estava vazio (ou tinha outras pessoas, mas não o `u001`). Aí entra o `par` novo.

inserir — passo 4 (método completo)

```
def inserir(self, chave, valor):
    indice = self._hash(chave)
    for par in self.baldes[indice]:
        if par[0] == chave:
            par[1] = valor
            return
    self.baldes[indice].append([chave, valor])
```

depois do passo 4

```
catalogo.inserir("u001", "João")
# baldes[2] agora: [["u001", "João"]]
```

Buscar `"u001"`:

## Passo 1. Calcular o *mesmo* índice.

A mesma chave *tem* que cair no mesmo andar. Por isso inserir e buscar usam a *mesma* `_hash`.

buscar — passo 1

```
def buscar(self, chave):
    indice = self._hash(chave)
```

## Passo 2. Percorrer só aquele balde.

buscar — passo 2

```
def buscar(self, chave):
    indice = self._hash(chave)
    for par in self.baldes[indice]:
        ...
```

## Passo 3. Se achar a chave → devolve o valor; senão → `None`.

buscar — passo 3 (método completo)

```
def buscar(self, chave):
    indice = self._hash(chave)
    for par in self.baldes[indice]:
        if par[0] == chave:
            return par[1]
    return None
```

Por que é rápido no caso médio? Porque a maioria dos baldes tem poucos itens — você quase não percorre a tabela toda.

## Colisão — o ponto que mais confunde

*Colisão* = duas chaves *diferentes* geram o *mesmo* índice. Isso *não é erro*: é esperado. Tratamento desta aula: *encadeamento (chaining)* — cada balde é uma listinha; as chaves colidentes ficam juntas no mesmo balde (lista da Aula 2 em miniatura).

- ▶ Chaves bem espalhadas, tabela folgada → busca média  $\approx O(1)$ .
- ▶ Quase tudo no mesmo balde → piora para  $O(n)$ .

### Atenção

Por isso o tamanho da tabela importa. Tabela pequena demais → mais colisões → mais lento. Tempo médio  $O(1)$  *assume* tabela folgada e hash espalhada.

## Comparação rápida (não misture com as outras aulas)

- ▶ Lista encadeada — inserir no início fácil; busca  $O(n)$ .
- ▶ Array por índice —  $O(1)$  se souber o índice; índice  $\neq$  CPF / id.
- ▶ Hash — busca por chave  $\approx O(1)$  médio; não mantém ordem; há colisões.
- ▶ BST (Aula 5) — mantém ordem + busca  $O(\log n)$ ; mais complexa.

No projeto final (logística): hash guarda `id_pacote` → `dados`; heap decide prioridade; grafo acha a rota. Hash acha a *ficha* do pacote — não decide quem sai primeiro (isso é heap).

## Exercício na mão (antes do código)

Use tabela de tamanho 5 e a função do curso: `indice = sum(ord(c) for c in chave) % 5` (veja `ord()`, `sum()` e `%` no glossário). Códigos úteis: `u=117`, `0=48`, `1=49`, `2=50`, `a=97`, `b=98`.

### Parte A — Calcular índices

Calcule o índice de `"u001"`, `"u002"`, `"ab"` e `"ba"`. Anote: chave | soma dos ord | soma % 5 | balde.

### Parte B — Desenhar a tabela

Insira `"u001"` → `"João"` e `"u002"` → `"Maria"`. Desenhe os 5 baldes e o conteúdo de cada um.

### Parte C — Forçar colisão

Ache duas chaves diferentes no mesmo balde. Sugestão: `"ab"` e `"ba"` têm os mesmos caracteres → mesma soma → colidem de propósito com esta função didática. Desenhe de novo o balde com dois pares.

### Parte D — Busca mental

1) Para buscar `"u001"`, quantos baldes você abre? 2) Se o balde tiver 2 pares, o que você compara? 3) Se o tamanho da tabela for 1, o que acontece com todas as inserções?

### Dica

Gabarito no Apêndice — olhe só *depois* de tentar no papel.

## Live coding: tabela com chaining

exemplos/04\_hash.py

```
class TabelaHash:
    def __init__(self, tamanho=8):
        self.tamanho = tamanho
        self.baldes = [[] for _ in range(tamanho)]

    def _hash(self, chave):
        return sum(ord(c) for c in str(chave)) % self.tamanho

    def inserir(self, chave, valor):
        indice = self._hash(chave)
        for par in self.baldes[indice]:
            if par[0] == chave:
                par[1] = valor
```

```

        return
    self.baldes[indice].append([chave, valor])

    def buscar(self, chave):
        indice = self._hash(chave)
        for par in self.baldes[indice]:
            if par[0] == chave:
                return par[1]
        return None

catalogo = TabelaHash(5)
catalogo.inserir("u001", "João Silva")
catalogo.inserir("u002", "Maria Souza")
print("Busca u001:", catalogo.buscar("u001"))
print("Busca u999:", catalogo.buscar("u999"))

```

### Boa prática

Ao inserir, primeiro percorra o balde: se a chave já existe, atualize o valor. Senão a mesma chave aparece duas vezes e a busca pega a antiga.

## Quando o prédio fica pequeno – crescer a tabela

`tamanho=8` no `__init__` é o tamanho *inicial*, não um teto. Encadeamento aguenta gente demais no mesmo andar, mas a listinha incha e a busca deixa de ser  $O(1)$ . O critério clássico é o *fator de carga*:

```
fator = quantidade_de_itens / tamanho_da_tabela
```

Quando o fator passa de um limite (nesta aula: `0.75`), dobramos os andares e o porteiro *recoloca todo mundo*. Isso se chama *rehash*: o resto muda, então o andar de "ab" em 5 andares ( $195 \% 5 = 0$ ) não é o mesmo em 10 ( $195 \% 10 = 5$ ). Sem recolocar, a busca iria ao andar errado.

**Passo 1.** Contar os itens: `self.quantidade` sobe só em inserção *nova* (atualizar não conta).

**Passo 2.** Depois do `append`, se `quantidade / tamanho > 0.75` → crescer.

**Passo 3.** `_crescer`: guardar os pares, `tamanho *= 2`, baldes novos, recolocar com `_hash`.

### Atenção

Não chame `inserir` de dentro de `_crescer`. `inserir` pode disparar outro crescimento. Recoloque os pares direto no balde novo.

exemplos/04\_hash\_dinamico.py

```

class TabelaHash:
    def __init__(self, tamanho=8, limite=0.75):

```

```

self.tamanho = tamanho      # tamanho *inicial* – pode dobrar
self.limite = limite        # critério: fator de carga
self.quantidade = 0
self.baldes = [[] for _ in range(tamanho)]

def _hash(self, chave):
    return sum(ord(c) for c in str(chave)) % self.tamanho

def fator_carga(self):
    return self.quantidade / self.tamanho

def inserir(self, chave, valor):
    indice = self._hash(chave)
    for par in self.baldes[indice]:
        if par[0] == chave:
            par[1] = valor
            return
    self.baldes[indice].append([chave, valor])
    self.quantidade += 1
    if self.fator_carga() > self.limite:
        self._crescer()

def _crescer(self):
    antigos = []
    for balde in self.baldes:
        antigos.extend(balde)
    self.tamanho *= 2
    self.baldes = [[] for _ in range(self.tamanho)]
    for chave, valor in antigos:
        indice = self._hash(chave)
        self.baldes[indice].append([chave, valor])

```

**Dica**

O `dict` do Python também cresce por fator de carga (bem mais sofisticado). Na Aula 4 o `04_hash.py` continua *fixo* de propósito: primeiro o fluxo, depois o extra de crescer.

## Práticas no computador

**Prática – Catálogo de usuários / cache**

Rode `python exemplos/04_hash.py`. Guarde `id` → nome (ou `url` → conteúdo). Busque por chave. Mude o tamanho para 5, imprima `catalogo.baldes` após cada inserção, force uma colisão e mostre os dois pares no mesmo balde. Explique em voz alta o que `_hash`, `inserir` e `buscar` fazem — sem olhar o código. Depois rode `python exemplos/04_hash_dinamico.py`: comece com tamanho 4 e veja o `tamanho` dobrar quando o fator passar de 0.75.

**Antes da próxima aula, você precisa conseguir**

- ▶ Por que hash existe, se já temos lista?

- ▶ O que significa `hash(chave) % tamanho`?
- ▶ O que é colisão e como o encadeamento resolve?
- ▶ Por que o tempo médio é  $O(1)$ , e quando isso falha?
- ▶ Em uma frase: diferença entre `dict` nativo e a `TabelaHash` da aula.
- ▶ O que é fator de carga e por que, ao crescer, é preciso recolocar as chaves?